AGRUPAMENTO DE ESCOLAS DE
CISTER - ALCOBAÇA

Ano letivo
2026/2027

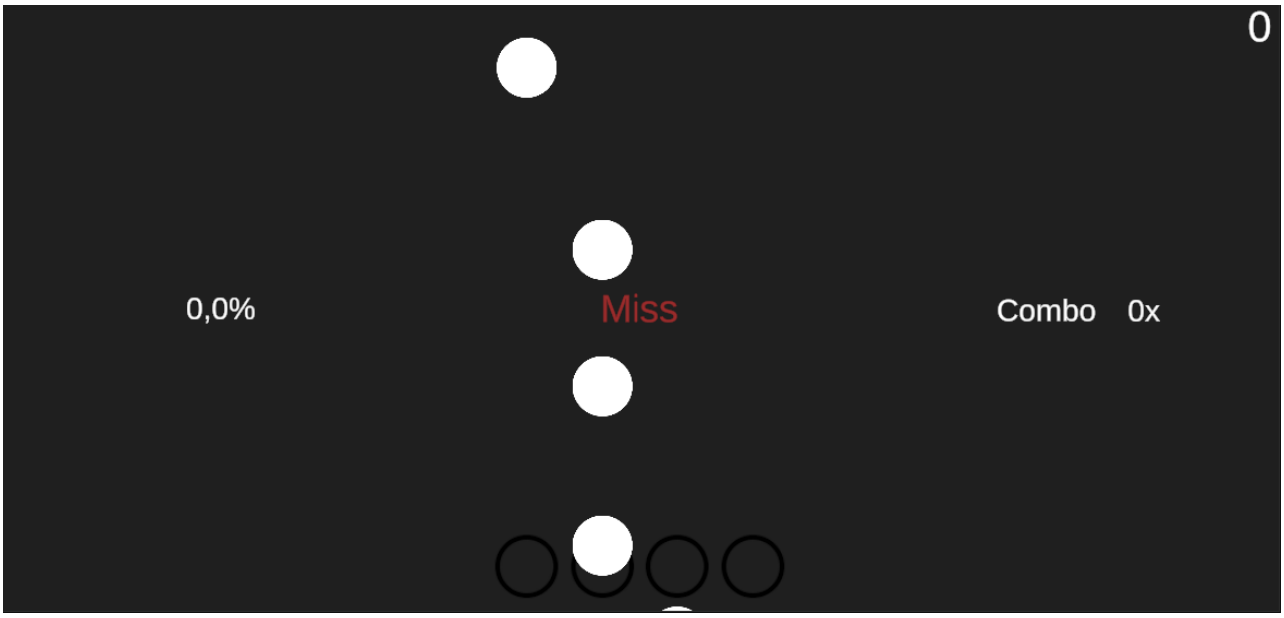
ESCOLA SECUNDÁRIA D. INÊS DE CASTRO

CURSO PROFISSIONAL DE

TÉCNICO DE GESTÃO E PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS

PROGRAMAÇÃO E SISTEMAS DE INFORMAÇÃO

SDJK



NOME DO FORMANDO: Gonçalo Andrade **Nº:**6

ANO: 3º **TURMA:** TGPSI

PESSOAS 2030
Os Fundos Europeus mais próximos de si.
PORTUGAL 2030
Cofinanciado pela União Europeia
unesco
Membro da Rede Escolas Associadas
SELO DE CONFORMIDADE EQAVET
PRR Plano de Recuperação e Resiliência

Índice

1. Introdução 3
2. Recursos utilizados 4
3. Desenvolvimento do projeto 6
   3.1. Introdução ao projeto 6
   3.2. Estrutura inicial do jogo 7
   3.3. Sistema de ritmo e sincronização musical 7
   3.4. Mecânicas de jogo e sistema de input 7
   3.5. Sistema de julgamento e precisão 8
   3.6. Sistema de pontuação, combo e resultados 9
   3.7. Sistema de Beatmaps 10
   3.8. Menus e definições 11
   3.9. Editor de Beatmaps 16
4. Dificuldades encontradas e como foram ultrapassadas 17
5. Conclusões 18
6. Referências 19

1. Introdução

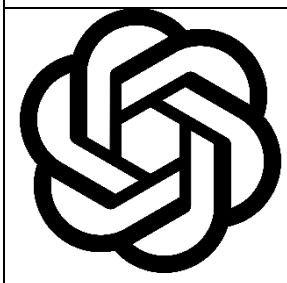
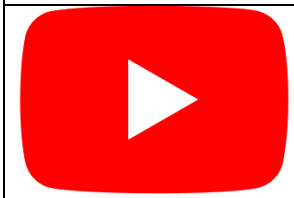
O presente relatório foi elaborado no âmbito dos módulos 17C e 16 da disciplina de Programação e Sistemas de Informação (PSI), lecionada pela professora Dária Ribeiro. No contexto destes módulos, foi proposto o desenvolvimento de um jogo que permitisse aplicar e aprofundar conhecimentos na aplicação Unity. Para responder a este desafio, foi desenvolvido o SDJK, um jogo de ritmo 2D criado com recurso ao Unity.

No SDJK, o jogador utiliza quatro teclas — S, D, J e K — para acertar notas que se deslocam por quatro colunas, acompanhando a música. A precisão de cada resposta é avaliada através das classificações Perfect, Great, Good e Miss, que influenciam a pontuação e o combo. O projeto inclui ainda menus, uma lista de músicas, um ecrã de resultados e um editor que permite alterar o BPM e o offset dos beatmaps.

O desenvolvimento teve como objetivo criar uma experiência de jogo funcional e agradável, aplicando conhecimentos de programação em C#, organização de cenas e elementos de interface, leitura de dados em formato JSON e sincronização entre a música e os elementos do jogo. Os jogos Quaver e osu!mania serviram de inspiração para algumas das mecânicas e opções de design adotadas.

Ao longo deste relatório são apresentados os recursos utilizados e as principais etapas de desenvolvimento do projeto. São descritos o sistema de sincronização musical, as mecânicas de jogo, os sistemas de julgamento, pontuação e beatmaps, bem como os menus e o editor. O relatório aborda ainda as dificuldades encontradas e as soluções aplicadas, terminando com uma conclusão sobre o trabalho realizado e uma lista das referências consultadas.

2. Recursos utilizados

Recurso	Características/versão	Funcionalidade
	Unity 6000.6.4f1	Desenvolvimento do jogo, dos assets e do HUD
Portátil		Hardware que foi usado para o uso das ferramentas e dos recursos todos que foram utilizados para o desenvolvimento deste projeto
	Visual Studio Code	Desenvolvimento dos scripts do jogo
	ChatGPT 6 Luna	Organização de ideias, pesquisa e correção de erros
	Youtube	Referências de jogos, ideias de scripts e música sem copyright

	GitHub	Repositório do jogo, relatórios diários das alterações e releases para amigos testarem e darem debug

3. Desenvolvimento do projeto

3.1. Introdução ao projeto

O SDJK é um jogo de ritmo que consiste em pressionar quatro teclas (S, D, J e K) em determinados momentos de uma música, de forma a obter pontuação. O desempenho do jogador é também avaliado através da precisão dos acertos e do sistema de combo, que contabiliza os acertos consecutivos. O jogo foi baseado em Quaver e Osu!mania (figuras 1 e 2), que são dois jogos que são idênticos que tomei como inspiração para a criação do SDJK.



Figura 1 Quaver



Figura 2 Osu!

3.2. Estrutura inicial do jogo

Inicialmente, comecei por criar a cena ***MusicPlay***, apresentada na Figura 3, onde decorre a jogabilidade principal. Esta cena é constituída por quatro colunas, cada uma correspondente a uma tecla. É nestas colunas que surgem as notas que o jogador tem de acertar nos momentos corretos. As notas deslocam-se de cima para baixo até chegarem à zona de acerto. Dependendo do momento em que o jogador pressiona a tecla correspondente, são atribuídos diferentes tipos de julgamento.

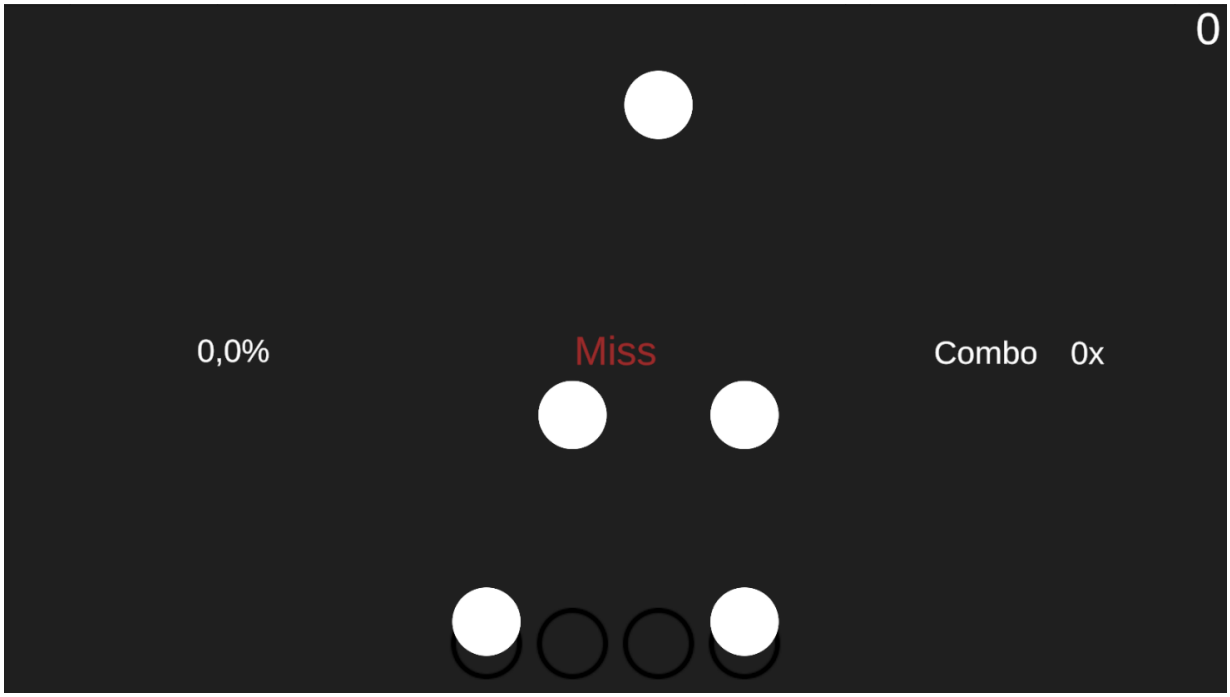


Figura 3 Cena MusicPlay

3.3. Sistema de ritmo e sincronização musical

O jogo utiliza um sistema baseado em AudioSettings.dspTime, que permite manter a sincronização com a música, evitando desvios no ritmo e garantindo uma temporização consistente ao longo da sua execução. Este sistema permite manter o BPM definido, assegurando que as notas surgem nos momentos corretos.

O AudioSettings.dspTime utiliza o tempo de funcionamento do sistema de áudio, medido em segundos, para calcular o tempo decorrido. Esta abordagem oferece uma maior precisão na sincronização musical quando comparada com a utilização de Time.deltaTime.

3.4. Mecânicas de jogo e sistema de input

Existem várias mecânicas no jogo, o julgamento, associar precisão, contar pontuação, ler resultados, mas a principal é o sistema de input. O input é feito somente com 4 teclas, sendo

essas o nome do jogo (S, D, J, K), as notas descem do topo da tela até à zona de acerto, marcada na figura 4, onde o jogador tem que então pressionar a tecla respetiva à coluna onde a nota se encontra e será depois avaliado pelo sistema de julgamento e precisão.

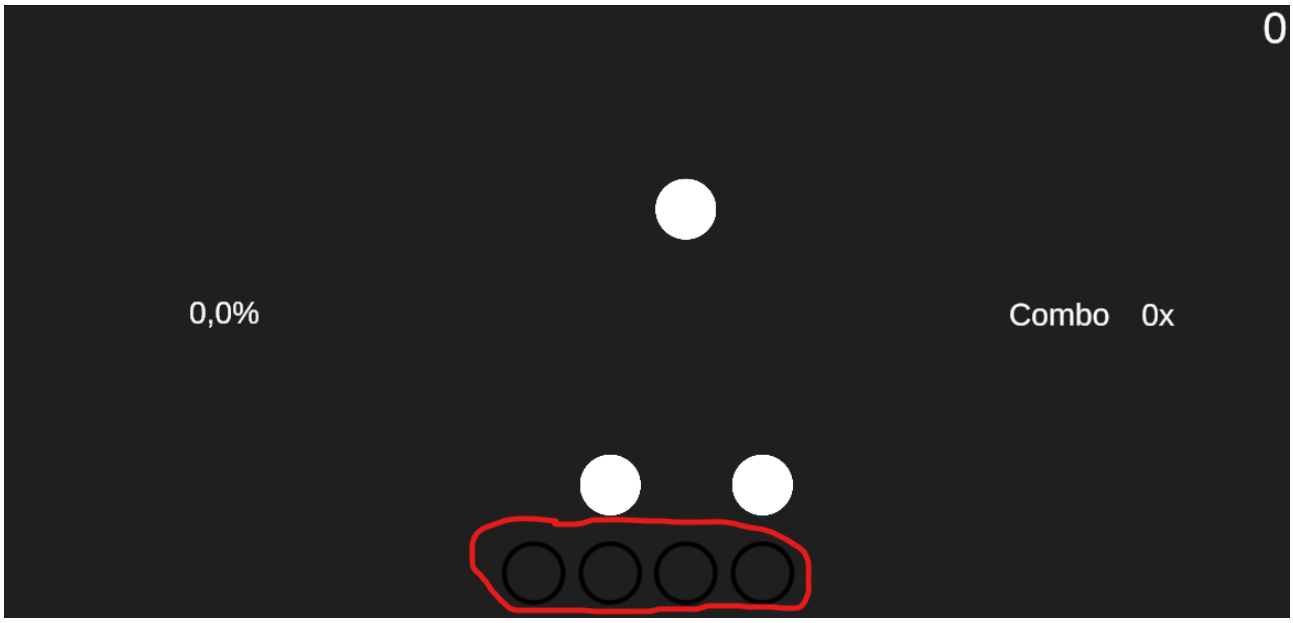


Figura 4 Zona de acerto

3.5. Sistema de julgamento e precisão

Os julgamentos são realizados através do script ***Judge.cs***, que contém uma função responsável por verificar a posição temporal das notas relativamente à zona de acerto. Essa diferença é posteriormente comparada com os thresholds, que definem os limites máximos para cada classificação: Perfect, Great, Good ou Miss.

Quando uma nota ultrapassa o limite definido para o acerto, é automaticamente classificada como Miss e removida do jogo.

Cada tipo de julgamento é apresentado no ecrã com uma cor específica: Perfect – azul, Great – verde, Good – amarelo e Miss – vermelho. Desta forma, o jogador consegue identificar facilmente a classificação de cada acerto. O nome do julgamento é apresentado através de uma animação de *pop*, que aumenta temporariamente o tamanho do texto antes de este desaparecer gradualmente.

Este script também é responsável por calcular a precisão do jogador, o combo atual e o combo máximo alcançado durante a partida. O combo atual é ainda utilizado como multiplicador da pontuação, permitindo aumentar os pontos obtidos à medida que o jogador consegue realizar acertos consecutivos.

No final da música, os dados relativos aos acertos, julgamentos, precisão, combo e pontuação são guardados para serem posteriormente utilizados na cena ***ResultsScreen***, apresentada na figura 5, onde são apresentados os resultados da partida.

Aprofundando um pouco mais o funcionamento do ***Judge.cs***, o cálculo da diferença temporal entre o momento em que o jogador pressiona a tecla e o momento em que a nota deveria ser acertada é realizado através dos thresholds. Estes valores, definidos como float (figura 5), representam as margens de tolerância em segundos relativamente ao momento exato do acerto, representado pelo valor zero. Quanto menor for a diferença em relação a zero, melhor será o julgamento atribuído.

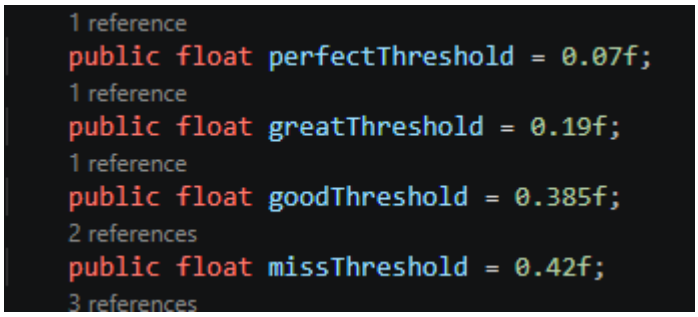


Figura 5 Thresholds

Através deste sistema também é calculada a precisão do jogador. Cada tipo de julgamento tem uma percentagem de precisão associada: Perfect – 100%, Great – 75%, Good – 50% e Miss – 0%. Estes valores são adicionados a uma variável que armazena a percentagem total acumulada. Posteriormente, esse valor é dividido pelo número de notas já julgadas, permitindo calcular a precisão atual do jogador.

3.6. Sistema de pontuação, combo e resultados

Na cena ***ResultsScreen*** são então apresentados todos os valores que foram guardados para representar o desempenho do jogador, pontuação final, combo máximo, precisão e os julgamentos são representados pelos pontos que dão e não pelos nomes, portanto em vez de aparecer Perfect mostra 300.

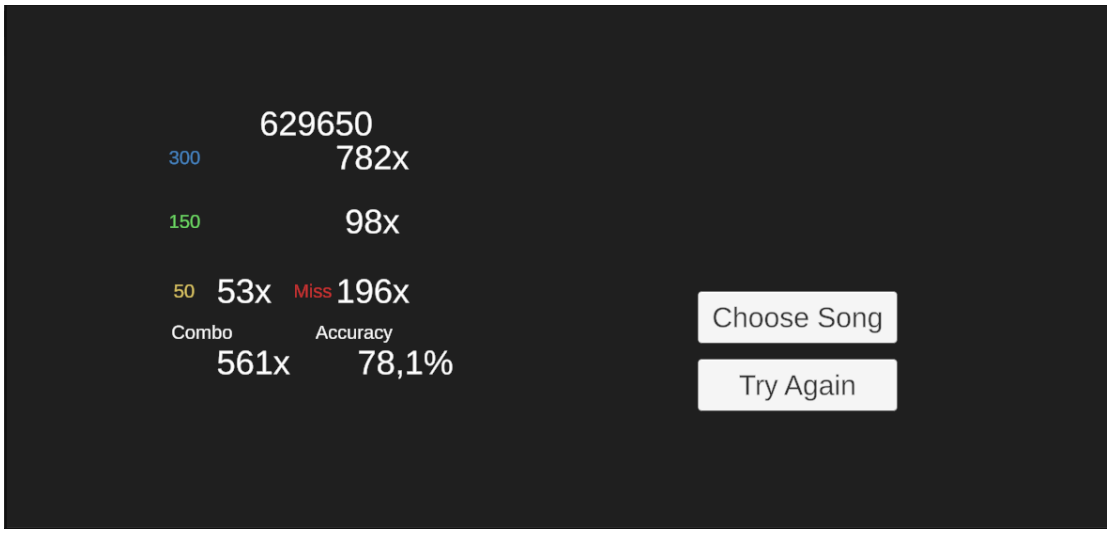


Figura 6 ResultsScreen

Os pontos são contados da seguinte forma, Perfect – 300, Great – 150, Good – 50 e Miss – 0, estes pontos são depois expostos a uma fórmula que usa o combo como multiplicador de pontuação para o jogador ser recompensado pelo o combo atual. A pontuação é guardada numa variável e transferida entre scripts através de uma classe estática que depois é usada e acessada na ***ResultsScreen*** para demonstrar os valores que são apresentados na figura acima.

3.7. Sistema de Beatmaps

Adicionei também uma maneira de criar "Beatmaps", que são os mapas do jogo, são criados em ficheiros JSON com uma estrutura específica para o jogo. Como é mostrado na figura 7 os JSON são criados com duas variáveis: noteBeat (o beat em que a nota aparece) e laneIndex (a coluna em que aparece de 0 a 3).

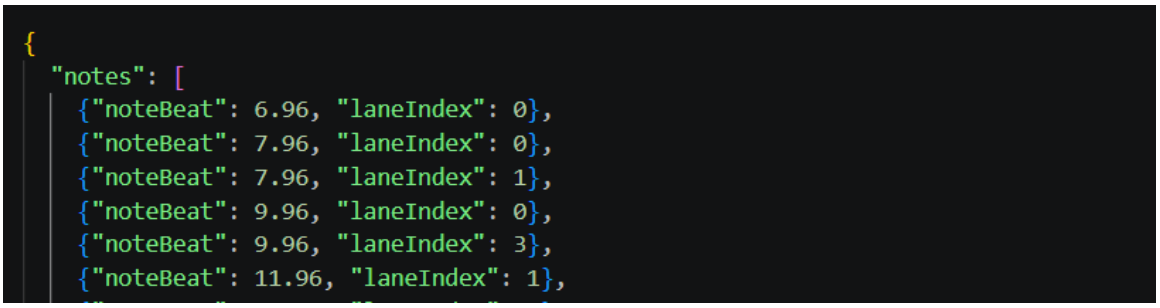


Figura 7 Estrutura do JSON do Beatmap

Este JSON é depois interpretado por vários scripts para ser guardado e usado posteriormente quando o jogador escolher o beatmap na cena BeatmapList, sendo passado de script em script para chegar ao Notespawner.cs onde é avaliado se falta alguma coisa para o bom funcionamento do beatmap ou se está tudo certo, depois o script Notespawner.cs lê os dados do ficheiro JSON e

cria as notas nos beats e nas colunas indicadas.

```
0 references | Unity Message
public void Update()
{
    if (beatList == null)
        return;

    if (nextBeatIndex < beatList.Count)
    {
        NoteData nextBeat = beatList[nextBeatIndex];
        Vector3 spawnPosition = new Vector3(lanePositions[nextBeat.laneIndex], 15f, 0f);

        if (conductor.songPositionInBeats >= nextBeat.noteBeat - spawnDistanceInBeats)
        {
            GameObject note = Instantiate(notePrefab, spawnPosition, Quaternion.identity);
            Note noteScript = note.GetComponent<Note>();
            noteScript.noteBeat = nextBeat.noteBeat;
            noteScript.laneIndex = nextBeat.laneIndex;
            noteScript.conductor = conductor;

            lanes[nextBeat.laneIndex].notes.Add(noteScript);
            nextBeatIndex++;
        }
    }
}
```

Figura 8 Notespawner.cs

3.8. Menus e definições

Criei uma cena nova para fazer de menu principal, fiz um menu super simples com o título do jogo e com 4 botões, Play, Edit, Settings e Exit. Todos os menus foram feitos com a mesma simplicidade de cores para não perder o tema de cores neutras. Também foi necessário ajustar as definições dos Canvas para tornar a interface utilizável em diferentes resoluções de ecrã.



Figura 9 MainMenu

Clicando no botão ***Play*** o jogador é levado para a cena ***BeatmapList***, demonstrada na figura 10, esta cena é dividida em dois panels, sendo o da esquerda onde aparece a lista dos beatmaps alinhada horizontalmente, elemento a elemento, e o painel da direita onde aparece a imagem e as informações de BPM, dificuldade, o criador do mapa, o nome do mapa e o criador da música, do beatmap que o jogador escolheu e também o botão Play para ir para a jogabilidade principal.

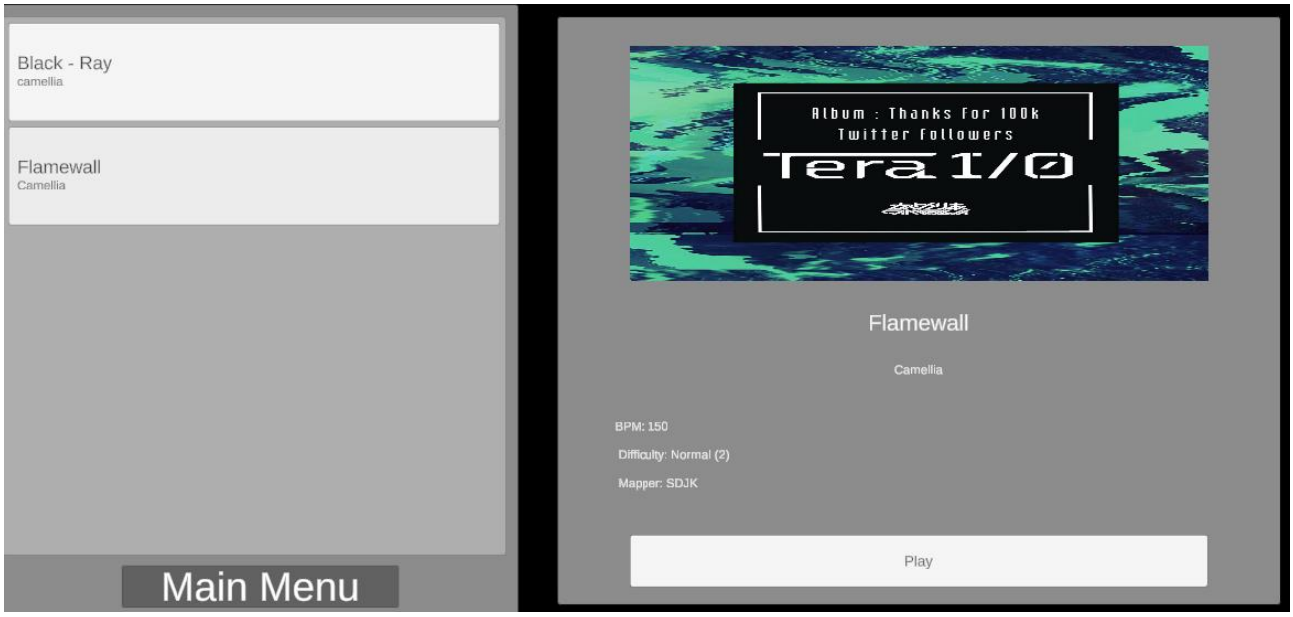


Figura 10 BeatmapList

Clicando em Play, na cena ***BeatmapList***, o jogador é levado para a cena ***MusicPlay*** onde acontece a jogabilidade principal. Na zona de acerto, que são sprites de círculos pretos, quando a cena carrega mostra as teclas correspondentes a cada coluna, como está representado na figura 11, e ao fim de um tempo essas letras desaparecem e fica somente os sprites dos círculos. Estes sprites têm uma animação (figura 12) quando o jogador pressiona a tecla correspondente.

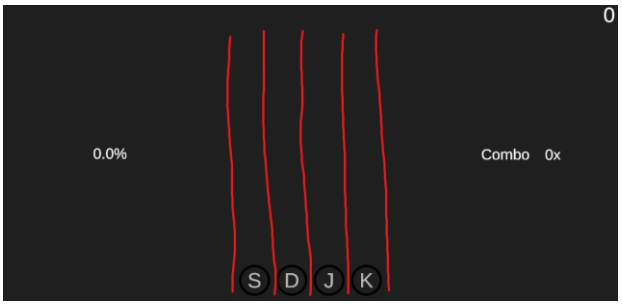


Figura 11 Colunas

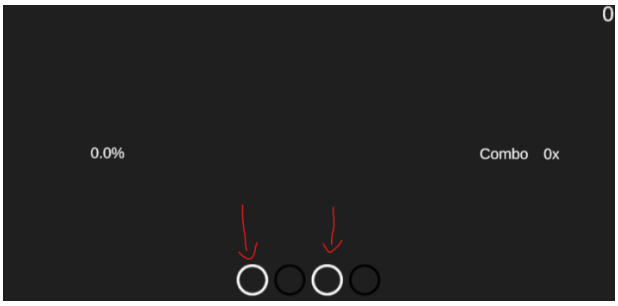


Figura 12 Animação

Ainda nesta cena o jogador tem um menu de pausa (figura 13) que é ativado após pressionar a tecla ***Esc*** no teclado, que pausa o jogo sem perder o ritmo e mostra três botões, Resume, Restart e Exit Song.

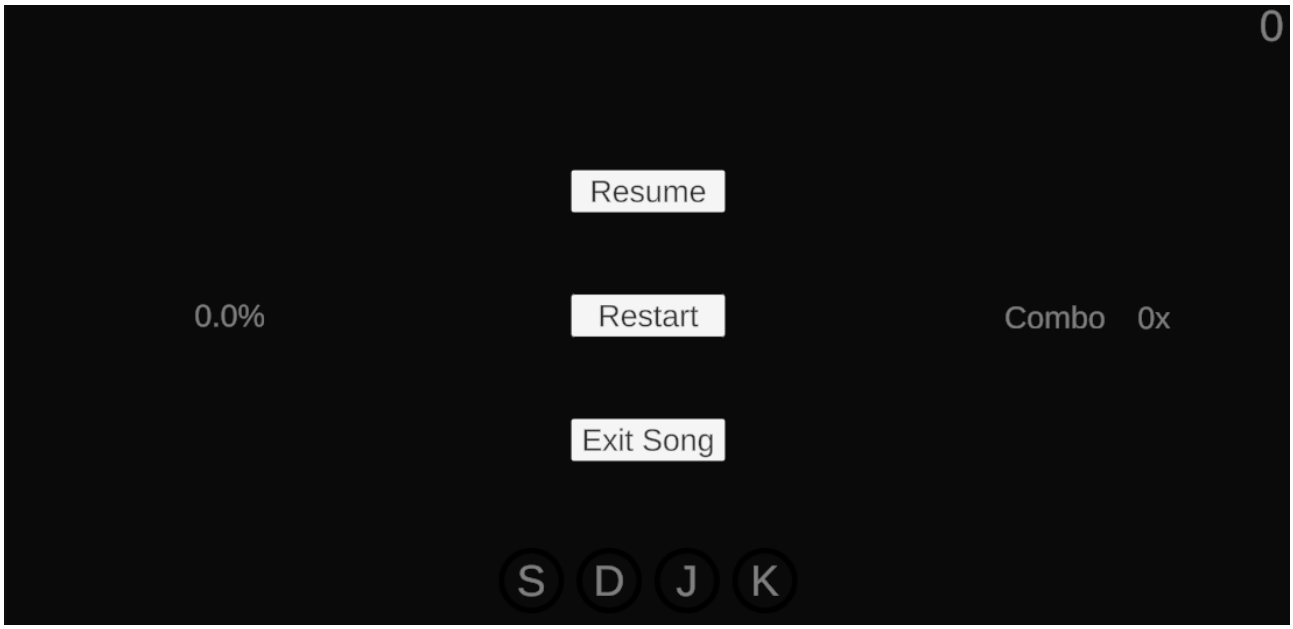


Figura 13 Menu de pausa

No fim da música o jogador é levado para a cena ***ResultsScreen (figura 14)***, onde vai ver o seu desempenho. Tem dois botões, um para ir de volta para a cena ***BeatmapList*** e escolher outra música, e o outro para tentar outra vez, voltando para a cena ***MusicPlay***. Os textos a cor são os julgamentos, não são alterados a qualquer momento, o que é alterado será os textos à sua frente, ficando por exemplo: 300 120x.

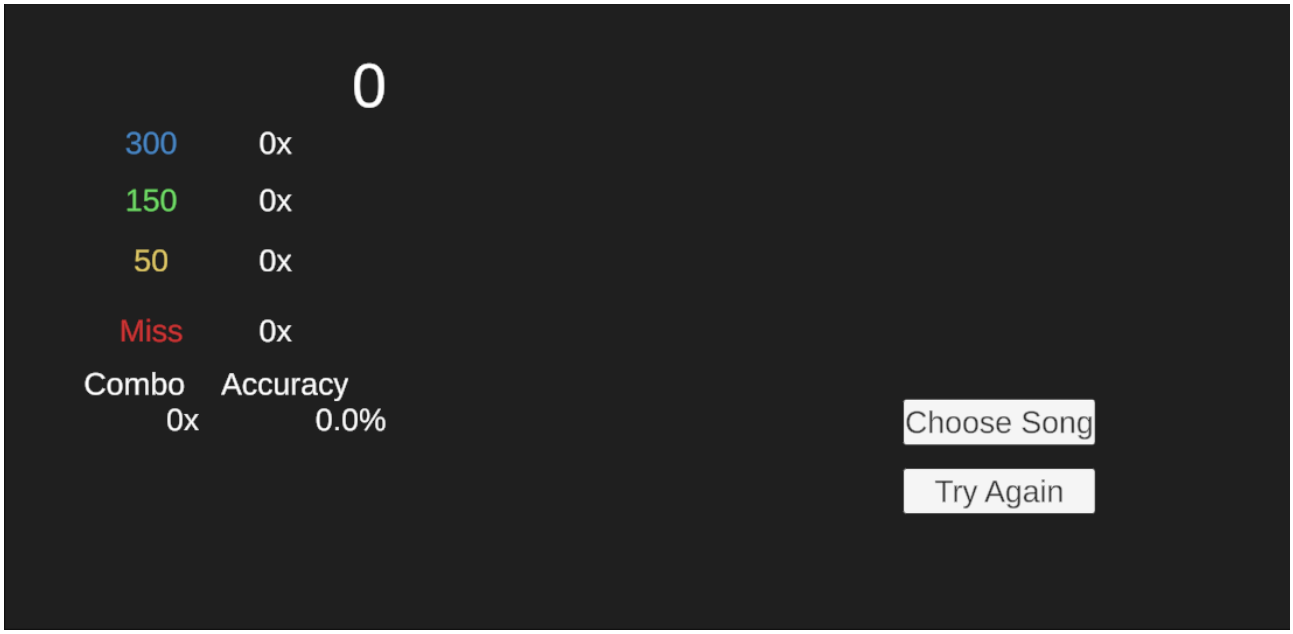


Figura 14 ResultsScreen

Voltando ao menu principal, se o jogador pressionar o botão ***Edit***, vai ser levado para a cena ***BeatmapEdit (figura 15)***. Novamente mantendo o tema de cores neutras com cinzento e branco, um botão para voltar para o menu principal e um botão para salvar as alterações. Tem

uma dropdown list que mostra todos os beatmaps. Na barra inferior tem 3 botões em branco que iam servir de play/pause, avançar e recuar, o texto ***Time*** ia mostrar a posição em segundos e o texto ***Beat*** ia mostrar a posição em beats, mas devido às dificuldades encontradas e ao tempo perdido com as mesmas, estas funcionalidades não foram concluídas.



Figura 15 BeatmapEdit

Se o jogador clicar no botão ***Settings*** é levado para a cena ***Settings (figura 16)***, onde só pode alterar o volume do jogo com um slider. Tem um botão para voltar para o menu principal e continua a usar as cores neutras do resto do jogo.

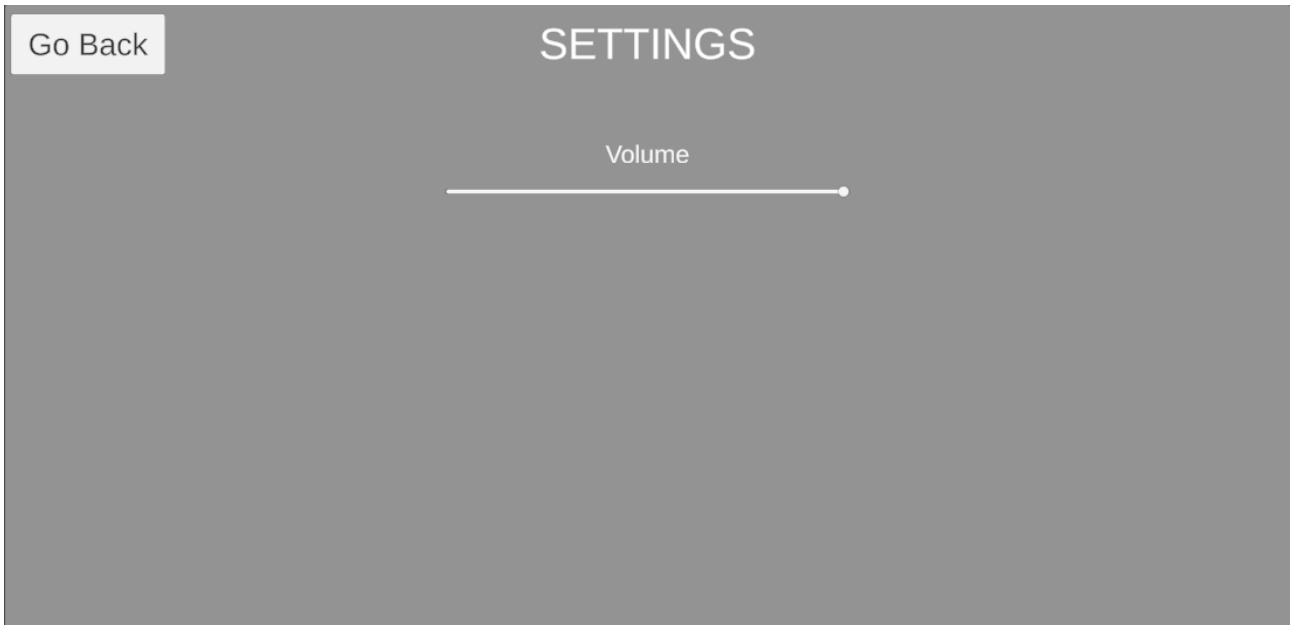


Figura 16 Settings

3.9. Editor de Beatmaps

A cena ***BeatmapEdit*** serve então para editar os beatmaps. O jogador consegue alterar o BPM do beatmap, não o da música, e consegue alterar o offset. O offset é o intervalo de tempo, em segundos, entre o início da reprodução do ficheiro de áudio e o primeiro beat relevante, estes ficheiros normalmente têm uns milissegundos até começar o primeiro beat da música e como é um jogo de ritmo cada milissegundo conta, então esse tempo precisa ser compensado com o offset.



Figura 17 BPM e OFFSET

Inicialmente esta cena ia servir para editar e criar beatmaps, portanto colocar o BPM, o offset e criar a posição das notas nos beats corretos, mas por dificuldades decidi simplificar e ficar só pela alteração do BPM e do offset.

Quando o jogador pressiona o botão save os valores são então guardados, num ficheiro JSON, com o seguinte nome: ***nomedamúsica_dificuldade.json*** e será guardado nesse ficheiro o nome do beatmap, a dificuldade, o BPM e o offset, como está representado na figura 18.

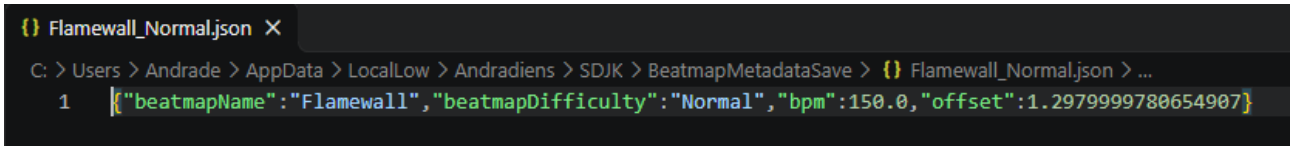


Figura 18 JSON de edit

4. Dificuldades encontradas e como foram ultrapassadas

A primeira dificuldade foi perceber como funcionava o sistema de jogos de ritmo, ultrapassei facilmente com pesquisas e com o script que um desenvolvedor criou que revolucionou todos os jogos de ritmo. Encontrei dificuldades também na criação dos julgamentos, para descobrir como funcionavam e como ia fazer os thresholds certos para cada um mas com ajuda de IA consegui resolver todos os problemas. Tive dificuldades também no desenvolvimento do menu de pausa, pesquisei bastante e não estava a conseguir realizar o objetivo de pausar o jogo por completo e depois retomar sem perder ritmo, mas com um pouco mais de pesquisa no Youtube consegui então terminar o menu de pausa.

Na cena ***BeatmapEdit*** tive um erro com o Vertical Layout Group, que serve para organizar os beatmaps todos na vertical sem ficarem uns em cima dos outros, os beatmaps não estavam a aparecer e estavam com um X vermelho na aba ***Scene***, depois de tanto analisar reparei que era um simples problema de posicionamento e tamanho dos elementos no canvas que estava a impedir de funcionar como era suposto.

A pontuação do jogador estava a ser calculada incorretamente. No começo a conta que fazia no ***Judge.cs*** era *score += data.score * combo*, fazendo com que a pontuação ficasse bastante extensa. Corrigi mudando a fórmula de cálculo para *score += data.score * (1+(combo/100))*, fazendo um incremento melhor para a pontuação final do jogador.

5. Conclusões

O desenvolvimento do SDJK permitiu criar um jogo de ritmo 2D funcional, com quatro colunas, um sistema de sincronização com a música, classificações de precisão e cálculo de pontuação e combo. Foram também desenvolvidos menus, uma lista de beatmaps, um ecrã de resultados e opções para ajustar o BPM e o offset dos mapas.

Ao longo do projeto, foram aprofundados conhecimentos de programação em C#, utilização do Unity, organização de elementos de interface e leitura de dados em formato JSON. A implementação do sistema de ritmo exigiu especial atenção à correspondência entre o tempo da música e o aparecimento das notas. As dificuldades encontradas nos sistemas de pausa, pontuação e organização da interface foram resolvidas através de pesquisa, consulta de documentação e testes.

Algumas funcionalidades inicialmente previstas para o editor de beatmaps, como a criação e edição direta das notas, não foram concluídas. Como trabalho futuro, seria possível desenvolver essas opções, melhorar a apresentação do editor e acrescentar mais beatmaps e definições. De um modo geral, o projeto cumpriu o objetivo de criar uma versão jogável e permitiu consolidar os conhecimentos aplicados durante o seu desenvolvimento.

6. Referências

FIZZD (s.d.). How to make a rhythm game: a quick and dirty guide to setting up your project. Disponível em: https://fizzd.me/posts/how-to-make-a-rhythm-game-a-quick-and-dirty-guide-to-setting-up-your-project. Acedido em [06/10/2026].

GRAHAM TATTERSALL (2019). Coding to the Beat: Under the Hood of a Rhythm Game in Unity. Game Developer, 15 de maio. Disponível em: https://www.gamedeveloper.com/audio/coding-to-the-beat---under-the-hood-of-a-rhythm-game-in-unity. Acedido em [06/10/2026].

UNITY TECHNOLOGIES (s.d.). Button — Unity User Manual. Disponível em: https://docs.unity3d.com/Packages/com.unity.ugui@2.7/manual/script-Button.html. Acedido em [19/10/2026].

UNITY TECHNOLOGIES (s.d.). UnityEvent.AddListener — Unity Scripting API. Disponível em: https://docs.unity3d.com/ScriptReference/Events.UnityEvent.AddListener.html. Acedido em [19/10/2026].

UNITY TECHNOLOGIES (s.d.). Unity 6 User Manual. Disponível em: https://docs.unity3d.com/6000.0/Documentation/Manual/index.html. Acedido em [08/10/2026].